

Randomized Algorithms

Random Walks on Undirected Graphs

Joseph Chuang-Chieh Lin

Department of Computer Science & Engineering,
National Taiwan Ocean University

Spring 2026



Outline

- 1 The random-walk model
- 2 Stationary distribution
- 3 Hitting and commute times
- 4 Cover time
- 5 Application: s - t connectivity



Why random walks on graphs?

- A random walk is a Markov chain whose states are vertices of a graph.
- It is a simple model of local exploration: from the current vertex, choose a random neighbor.
- In this lecture note, we use graph structure to bound:
 - stationary distributions,
 - hitting and commute times,
 - cover times,
 - and a randomized low-space algorithm for s - t connectivity.

Guiding question

How long does local random exploration need to see a target vertex, or even all vertices?



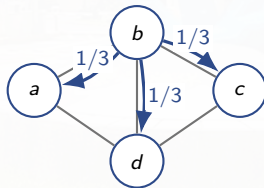
Random walk on an undirected graph

Let $G = (V, E)$ be a finite undirected graph.

Definition 1

At vertex u , the standard random walk moves to a uniformly chosen neighbor:

$$P_{u,v} = \begin{cases} 1/d(u), & (u,v) \in E, \\ 0, & \text{otherwise.} \end{cases}$$



The transition matrix is local

- The transition matrix P has one row per vertex.
- Row u has nonzero entries only at neighbors of u .
- The degree $d(u)$ controls the transition probabilities leaving u .

Example

If $N(u) = \{v_1, v_2, v_3\}$, then

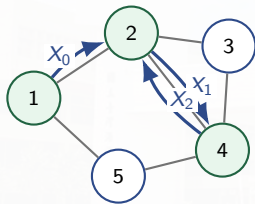
$$P_{u,v_1} = P_{u,v_2} = P_{u,v_3} = \frac{1}{3}.$$

Memoryless property

Given the current vertex, the previous trajectory carries no additional information for choosing the next vertex.



A sample walk



A walk is a random sequence

$$X_0, X_1, X_2, \dots$$

with

$$\Pr[X_{t+1} = v \mid X_t = u] = P_{u,v}.$$

Note

The graph is fixed; the randomness is in the walk.

Structural assumptions

In the following discussions, we assume G is:

- finite,
- undirected,
- connected,
- non-bipartite.

Why?



Structural assumptions

In the following discussions, we assume G is:

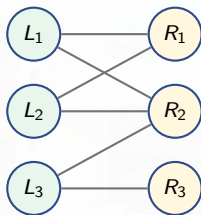
- finite,
- undirected,
- connected,
- non-bipartite.

Why?

- Connected $\Rightarrow$ every vertex can be reached from every other vertex.
- Non-bipartite $\Rightarrow$ the walk is aperiodic.
- Together, the *finite* Markov chain converges to a stationary distribution.



Bipartite graphs cause period two



bipartite: alternate sides

In a bipartite graph, every step switches side.

$$L \rightarrow R \rightarrow L \rightarrow R \rightarrow \dots$$

So a return to the starting vertex can happen only at even times.

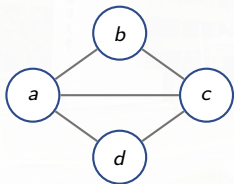
Consequence

The period is 2, so the chain is not aperiodic.

Non-bipartite means aperiodic

Lemma 1

A random walk on an undirected graph G is aperiodic if and only if G is not bipartite.



odd cycle present

- Every undirected edge gives a return of length 2: $v \rightarrow u \rightarrow v$.
- An odd cycle gives an odd return length.
- Having return lengths with gcd 1 means period 1 (i.e., *aperiodic*).

The stationary distribution

Theorem 1

For a finite, connected, non-bipartite undirected graph,

$$\pi_v = \frac{d(v)}{2|E|}$$

is the stationary distribution of the simple random walk.



The stationary distribution

Theorem 1

For a finite, connected, non-bipartite undirected graph,

$$\pi_v = \frac{d(v)}{2|E|}$$

is the stationary distribution of the simple random walk.

The walk spends more time at high-degree vertices, because they are incident to more directed edge-ends.



The stationary distribution

Theorem 1

For a finite, connected, non-bipartite undirected graph,

$$\pi_v = \frac{d(v)}{2|E|}$$

is the stationary distribution of the simple random walk.

The walk spends more time at high-degree vertices, because they are incident to more directed edge-ends.

The “distribution” is valid:



The stationary distribution

Theorem 1

For a finite, connected, non-bipartite undirected graph,

$$\pi_v = \frac{d(v)}{2|E|}$$

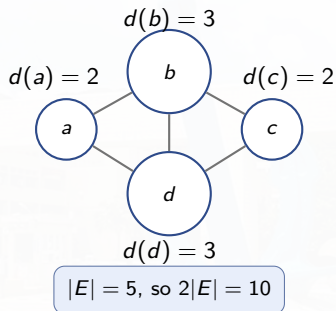
is the stationary distribution of the simple random walk.

The walk spends more time at high-degree vertices, because they are incident to more directed edge-ends.

The “distribution” is valid: $\sum_{v \in V} \pi_v = \sum_{v \in V} \frac{d(v)}{2|E|} = 1.$



Stationary distribution: a concrete picture



$$\pi_a = \pi_c = \frac{2}{10}, \quad \pi_b = \pi_d = \frac{3}{10}.$$

Why the formula works: detailed balance

For every edge $(u, v) \in E$,

$$\pi_u P_{u,v} = \frac{d(u)}{2|E|} \cdot \frac{1}{d(u)} = \frac{1}{2|E|}.$$

Similarly,

$$\pi_v P_{v,u} = \frac{1}{2|E|}.$$

Therefore

$$\pi_u P_{u,v} = \pi_v P_{v,u}.$$

Meaning

In stationarity, the probability flow across each undirected edge is balanced in both directions.

Hitting time, commute time, cover time

Definition 2

For vertices $u, v \in V$, let

$$h_{u,v} = \mathbb{E}[\text{time to first reach } v \mid X_0 = u].$$

This is the *hitting time* from u to v .

Two related quantities

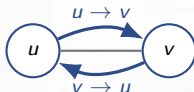
- **Commute time:**

$$h_{u,v} + h_{v,u}.$$

- **Cover time:** maximum expected time, over starting vertices, to visit every vertex.



Directed-edge viewpoint



Replace each undirected edge by two directed edges.

- There are $2|E|$ directed edges.
- The edge traversed at time t is a Markov-chain state.
- In stationarity, each directed edge has probability $1/(2|E|)$.

Lemma 7.14: adjacent commute time

Lemma 2

If $(u, v) \in E$, then

$$h_{u,v} + h_{v,u} \leq 2|E|.$$



Lemma 7.14: adjacent commute time

Lemma 2

If $(u, v) \in E$, then

$$h_{u,v} + h_{v,u} \leq 2|E|.$$

Core idea

The expected return time to a directed edge $u \rightarrow v$ is

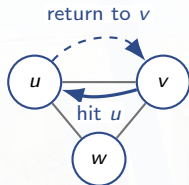
$$\frac{1}{1/(2|E|)} = 2|E|.$$

Returning to the directed edge $u \rightarrow v$ requires going from v to u and then back to v through that edge.



Why this bounds the whole commute

Suppose the walk has just traversed $u \rightarrow v$, so it is currently at v .



specific return via $u \rightarrow v$ is only one possible way

$$h_{v,u} + h_{u,v} \leq \mathbb{E}[\text{additional time until next traversal of } u \rightarrow v] = 2|E|.$$

A useful caution

Why the sum appears

The return time to a directed edge controls a complete round trip:

$$V \rightsquigarrow U \rightsquigarrow V.$$

The first traversal has already occurred; the bound concerns the additional time until the next one.



Cover time

Definition 3

The cover time C_G is

$$C_G = \max_{v \in V} \mathbb{E}[\text{time to visit all vertices} \mid X_0 = v].$$

Goal

Bound C_G using simpler quantities such as:

- number of edges $|E|$,
- number of vertices $|V|$,
- maximum hitting time $\max_{u \neq v} h_{u,v}$.



Spanning-tree tour bound

Lemma 3

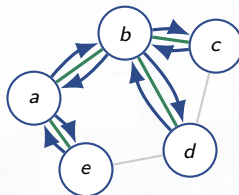
For a connected undirected graph $G = (V, E)$,

$$C_G \leq 2|E|(|V| - 1).$$

- Choose any spanning tree T of G .
- A depth-first tour crosses each tree edge once in each direction.
- Each tree-edge commute costs at most $2|E|$ by Lemma 2.
- There are $|V| - 1$ tree edges.



Spanning tree tour picture



DFS tour of spanning tree: each tree edge twice

$$\sum_{(x,y) \in T} (h_{x,y} + h_{y,x}) \leq 2|E|(|V| - 1).$$

Matthews' theorem

Lemma 4

Let G have n vertices. Then

$$C_G \leq H(n-1) \max_{u,v \in V: u \neq v} h_{u,v},$$

where

$$H(n-1) = \sum_{i=1}^{n-1} \frac{1}{i}.$$

Matthews' theorem

Lemma 4

Let G have n vertices. Then

$$C_G \leq H(n-1) \max_{u,v \in V: u \neq v} h_{u,v},$$

where

$$H(n-1) = \sum_{i=1}^{n-1} \frac{1}{i}.$$

Why and when is it useful?

If all pairwise hitting times are small, the cover time is like coupon collector with hitting-time cost.



Matthews' theorem: proof idea

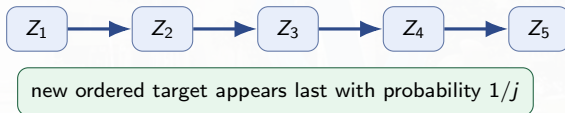
Let

$$B = \max_{u \neq v} h_{u,v}.$$

Choose a uniformly random ordering of the vertices:

$$Z_1, Z_2, \dots, Z_n.$$

Let T_j be the first time by which all of $Z_1, Z_2, \dots, Z_j$ have been visited.



For $j \geq 2$, the extra waiting interval satisfies

$$\mathbb{E}[T_j - T_{j-1}] \leq \frac{B}{j}.$$

Summing gives a harmonic factor.

Lemma 3 vs. Lemma 4

	Lemma 3	Lemma 4
Input needed	$ E $ and $ V $	maximum hitting time
Bound	$2 E (V - 1)$	$H(n - 1) \max_{u \neq v} h_{u,v}$
Method	spanning-tree tour	random ordering / coupon collector
Best when	no hitting-time info	good hitting-time estimates exist

Teaching point

Lemma 3: coarse but always available.

Lemma 4: can be sharper when the graph mixes or spreads quickly.



s - t connectivity with tiny memory

Given an undirected graph $G = (V, E)$ and two vertices s, t , decide whether s and t are connected.

- Breadth-first search or depth-first search solves this in linear time.
- But such **deterministic** graph search needs $\Theta(n)$ space to **remember visited vertices**.
 - Can be done in $O(n + m)$ time by BFS/DFS.
- The random-walk algorithm **uses only the current vertex** and a **step counter**.

Goal

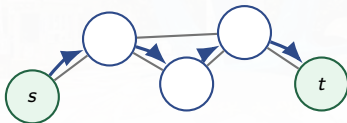
Trade time and error probability for very **small space**: $O(\log n)$ bits.



Algorithm 1: Testing connectivity by random-walks

s - t connectivity algorithm

- 1 Start a random walk from s .
- 2 If the walk reaches t within $2n^3$ steps, return “connected.”
- 3 Otherwise return “not connected.”



Correctness and one-sided error

- If there is no path from s to t , the walk can never reach t .
- Therefore returning “connected” is always correct.
- The only possible error is saying “not connected” even though a path exists.

Bounding the error

If s and t are connected, the expected time to reach t from s is at most the cover time of their component.

$$\mathbb{E}[T_{s \rightarrow t}] \leq C_G \leq 2nm < n^3.$$

Then Markov's inequality gives

$$\Pr[T_{s \rightarrow t} > 2n^3] \leq \frac{1}{2}.$$

Space usage and amplification

- Store current vertex: $O(\log n)$ bits.
- Store step counter up to $2n^3$: $O(\log n)$ bits.
- Choose a random neighbor at each step.

Amplification

Repeat the random walk independently r times.

$$\Pr[\text{"false negative after } r \text{ runs"}] \leq 2^{-r}.$$

The space remains $O(\log n)$; the time grows by a factor of r .



Takeaways (1/2)

- Random walks on connected non-bipartite undirected graphs converge to

$$\pi_v = \frac{d(v)}{2|E|}.$$

- Directed-edge stationarity gives the adjacent commute-time bound

$$h_{u,v} + h_{v,u} \leq 2|E|.$$

- Cover time can be bounded by either spanning-tree tours or Matthews' theorem.



Takeaways (2/2)

- The random-walk s - t connectivity algorithm is simple, one-sided, and uses only $O(\log n)$ space.
- In fact, Reingold's theorem later showed that even the randomization can be removed. [Omer Reingold J. ACM 2008]



Discussions

